MULTI-CRITERIA LIBRARY RESOURCE DISTRIBUTION

Python Project Report

1. Problem Statement

Develop a Python program that analyses book availability, borrowing demand and branch-wise requirements. It allocates copies according to supplied priority rules and calculates unmet demand for each branch and book title.

2. Project Overview

The program uses lists, tuples and dictionaries to store branch requests, available book copies and branch priorities. For each title, it allocates available copies to branches in priority order, then reports allocated copies and remaining unmet demand.

Submitted as a Python programming project

3. Objectives

• Record book availability for each title.
• Store borrowing demand for each library branch.
• Apply the given branch priority rules.
• Allocate copies without exceeding available stock.
• Calculate unmet demand for each branch and title.
• Summarise total demand, allocated copies and unmet demand.

4. Assumptions

• The example contains three branches and three book titles.
• Each branch has a stated demand for each title.
• Higher priority numbers are served first.
• If priorities tie, the branch with higher demand is served first; branch name breaks any remaining tie.
• Copies are whole numbers, and allocation cannot exceed demand or stock.
• The example demonstrates a supplied priority policy; it does not optimise all possible policies.

5. Algorithm

1. Define branches, book demands, available copies and priorities.
2. Create empty allocation and unmet-demand records.
3. Process each book title separately.
4. Sort branch requests by priority, then demand.
5. Allocate the smaller of requested copies and remaining stock.
6. Calculate unmet demand as requested minus allocated copies.
7. Calculate book-wise and overall totals.
8. Display allocation and validation results.

6. Flowchart

START
↓
Read stock, branch demand and priorities
↓
Select a book title
↓
Sort branches by priority
↓
Allocate copies and calculate unmet demand
↓
Repeat for every book title
↓
Display branch-wise and total results
↓
STOP

7. Pseudocode

```
BEGIN
  INPUT book availability, branch demands and priorities
  FOR each book title:
      remaining = available copies
      SORT branches by descending priority and demand
      FOR each branch:
          allocated = MIN(requested, remaining)
          unmet = requested - allocated
          remaining = remaining - allocated
      CALCULATE total demand, allocation and unmet demand
  DISPLAY branch-wise and book-wise results
END
```

8. Python Source Code

Save the program as **multi_criteria_library_distribution.py** and run it in VS Code.

```
# Multi-Criteria Library Resource Distribution
# Allocate available book copies using demand and priority rules.

branches = [
    ("Central", {"Python": 8, "DBMS": 6, "Data Structures": 5}),
    ("North",   {"Python": 5, "DBMS": 7, "Data Structures": 4}),
    ("South",   {"Python": 6, "DBMS": 4, "Data Structures": 8}),
]

# Available copies for each book title.
book_availability = {
    "Python": 12,
    "DBMS": 10,
    "Data Structures": 9,
}

# Higher priority value means the branch has higher priority.
# Priority order: Central > North > South.
branch_priority = {"Central": 3, "North": 2, "South": 1}

def allocate_books(branches, availability, priorities):
    allocations = {branch: {} for branch, _ in branches}
    unmet = {branch: {} for branch, _ in branches}
    summary = {}

    for title, total_copies in availability.items():
        remaining = total_copies
        requests = []
        for branch, demand in branches:
            requests.append((priorities[branch], branch, demand.get(title, 0)))
        # Sort by priority first; if tied, larger demand comes first.
        requests.sort(key=lambda item: (-item[0], -item[2], item[1]))

        for _, branch, requested in requests:
            assigned = min(requested, remaining)
            allocations[branch][title] = assigned
            unmet[branch][title] = requested - assigned
            remaining -= assigned

        total_demand = sum(demand.get(title, 0) for _, demand in branches)
        total_allocated = sum(allocations[b][title] for b, _ in branches)
        total_unmet = sum(unmet[b][title] for b, _ in branches)
        summary[title] = {
            "available": total_copies,
            "demand": total_demand,
            "allocated": total_allocated,
            "unmet": total_unmet,
            "leftover": remaining,
        }
    return allocations, unmet, summary

def main():
    allocations, unmet, summary = allocate_books(
        branches, book_availability, branch_priority
    )
    total_demand = sum(item["demand"] for item in summary.values())
    total_allocated = sum(item["allocated"] for item in summary.values())
    total_unmet = sum(item["unmet"] for item in summary.values())

    print("=" * 76)
    print("          MULTI-CRITERIA LIBRARY RESOURCE DISTRIBUTION")
    print("=" * 76)
    print("Priority: Central (3) > North (2) > South (1)")
    print("\nBOOK-WISE AVAILABILITY AND DEMAND")
    print("-" * 76)
    print(f"{'Book Title':<20}{'Available':>11}{'Demand':>10}{'Allocated':>12}{'Unmet':>9}")
    print("-" * 76)
    for title, item in summary.items():
        print(f"{title:<20}{item['available']:>11}{item['demand']:>10}"
              f"{item['allocated']:>12}{item['unmet']:>9}")

    print("\nBRANCH-WISE ALLOCATION")
    print("-" * 76)
    print(f"{'Branch':<12}{'Book Title':<20}{'Demand':>9}{'Allocated':>12}{'Unmet':>9}")
    print("-" * 76)
    for branch, demands in branches:
        for title, requested in demands.items():
            print(f"{branch:<12}{title:<20}{requested:>9}"
                  f"{allocations[branch][title]:>12}{unmet[branch][title]:>9}")

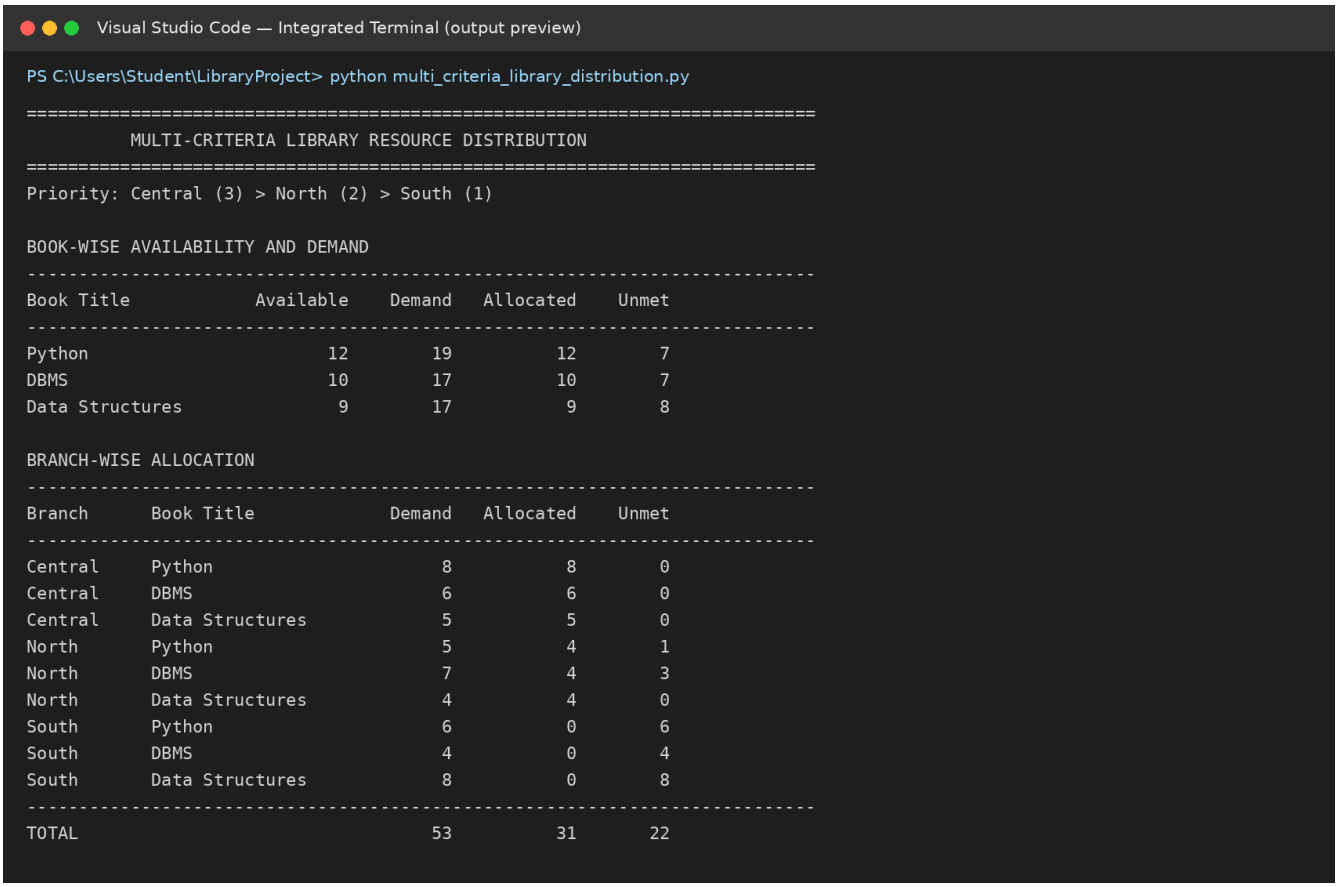
    print("-" * 76)
    print(f"{'TOTAL':<32}{total_demand:>9}{total_allocated:>12}{total_unmet:>9}")
    print("\nVALIDATION")
    print("  [OK] Allocations do not exceed available copies.")
    print("  [OK] Branch allocations do not exceed requested demand.")
    print("  [OK] Priority rules were applied to each book title.")
    print(f"  Total demand: {total_demand} | Allocated: {total_allocated} | Unmet demand: {total_unmet}")
    print("\nDistribution analysis completed.")

if __name__ == "__main__":
```

```
main()
```

9. Output Screenshot

The image below is a terminal-style preview of the output generated by the Python program. It resembles the VS Code integrated terminal but is not a direct screenshot captured from the VS Code application.



10. Test Cases

Test case	Expected result
Stock is enough for all branches	Unmet demand is zero for that title.
Stock is lower than total demand	Priority order is applied; remaining demand is reported as unmet.
A branch requests zero copies	Allocation and unmet demand are both zero.
Availability is zero	No copies are allocated; all requests remain unmet.
All titles processed	Book-wise and overall totals are displayed.

11. Conclusion

The program analyses book stock and branch-wise demand, applies priority rules, allocates available copies and calculates unmet demand. The summaries help library staff understand which requests can be fulfilled and where shortages remain.